

EnviroGuard v2

Frontend API integration — all functions that call API Gateway and handle responses

Structure	src/services/ — one file per domain. Each file exports typed functions the screens call.
Base helper	src/utls/api.ts — all fetches go through apiGet() and apiPost(). Never use fetch() directly in screens.
Fake data	Each service function reads from src/data/fake/ when USE_FAKE_DATA=true in env. Switch is one line.
Where used	Screens import from services/, never from api.ts directly. Services own the data shape translation.

1 — Base layer — api.ts

Every API call in the app goes through these two functions. They attach the base URL and JWT header automatically. Screens never call `fetch()` directly.

src/utils/api.ts

```
import * as SecureStore from 'expo-secure-store'; const BASE =
process.env.EXPO_PUBLIC_API_GATEWAY_URL; async function getToken() { return await
SecureStore.getItemAsync('jwt_token'); } export async function apiGet(path: string, params?: Record):
Promise { const token = await getToken(); const qs = params ? '?' + new
URLSearchParams(params).toString() : ''; const res = await fetch(`${BASE}${path}${qs}`, { headers: {
Authorization: `Bearer ${token}` } }); if (!res.ok) throw new Error(`${res.status} ${path}`); return
res.json(); } export async function apiPost(path: string, body: object): Promise { const token = await
getToken(); const res = await fetch(`${BASE}${path}`, { method: 'POST', headers: { 'Content-Type':
'application/json', Authorization: `Bearer ${token}` }, body: JSON.stringify(body) }); if (!res.ok)
throw new Error(`${res.status} ${path}`); return res.json(); } export async function apiPut(path:
string, body: object): Promise { const token = await getToken(); const res = await
fetch(`${BASE}${path}`, { method: 'PUT', headers: { 'Content-Type': 'application/json', Authorization:
`Bearer ${token}` }, body: JSON.stringify(body) }); if (!res.ok) throw new Error(`${res.status}
${path}`); return res.json(); }
```

src/constants/env.ts

```
export const USE_FAKE_DATA = process.env.EXPO_PUBLIC_USE_FAKE_DATA === 'true'; export const
ANTHROPIC_API_KEY = process.env.EXPO_PUBLIC_ANTHROPIC_API_KEY ?? ''; // .env file: //
EXPO_PUBLIC_API_GATEWAY_URL=https://abc123.execute-api.us-east-1.amazonaws.com/v1 //
EXPO_PUBLIC_ANTHROPIC_API_KEY=sk-ant-... // EXPO_PUBLIC_USE_FAKE_DATA=true ← flip to false when
backend is ready
```

2 — Map service — mapService.ts

Used by: MapScreen (on mount + every 15 min), bottom sheet Claude summary.

src/services/mapService.ts

```
import { apiGet } from '../utils/api'; import { USE_FAKE_DATA } from '../constants/env'; import {
fakeMapData } from '../data/fake/fakeMapData'; export interface MapLayer { noise: { index: number;
complaint_count_24h: number }; air: { aqi: number; pm25: number; o3: number; health_category: string
}; litter: { complaint_count_24h: number; avg_severity: number }; pollen: { grass: number; tree:
number; weed: number; total_index: number }; general: { report_count_24h: number }; } export interface
MapZone { neighborhood_id: string; name: string; lat: number; lng: number; composite_score: number;
severity: 'low' | 'moderate' | 'high' | 'very_high'; mode: 'api' | 'community'; confidence: 'low' |
'medium' | 'high'; layers: MapLayer; last_updated: string; } export interface RiskScore {
neighborhood_id: string; composite_score: number; severity: 'low' | 'moderate' | 'high' | 'very_high';
mode: 'api' | 'community'; confidence: 'low' | 'medium' | 'high'; breakdown: { air: number; noise:
number; pollen: number; litter: number; general: number }; } // Primary map call – used by MapScreen
on mount and every 15 min. // ALWAYS call this regardless of mode. Never call /posts directly from the
frontend. // mode='api' → Lambda calls AirNow, OpenWeather, Ambee, 311 internally // mode='community'
→ Lambda aggregates posts table internally // Same endpoint, same frontend call, Lambda handles the
difference. export async function fetchMapData( lat: number, lng: number, mode: 'api' | 'community' ):
Promise { if (USE_FAKE_DATA) return fakeMapData; return apiGet('/map-data', { lat: lat.toString(),
lng: lng.toString(), mode }); } // Risk score – called when the bottom sheet needs just the composite
score // without refetching all map layers. EC2 FastAPI computes it. export async function
fetchRiskScore( neighborhood: string, mode: 'api' | 'community' ): Promise { if (USE_FAKE_DATA) return
{ neighborhood_id: neighborhood, composite_score: 67, severity: 'high', mode, confidence: 'high',
breakdown: { air: 85, noise: 72, pollen: 68, litter: 45, general: 20 } }; return
apiGet(`/risk-score/${neighborhood}`, { mode }); } // How MapScreen uses these: // // useEffect(() =>
{ // fetchMapData(location.lat, location.lng, mode).then(setMapData); // const poll = setInterval(()
=> // fetchMapData(location.lat, location.lng, mode).then(setMapData), // 15 * 60 * 1000 // ); //
return () => clearInterval(poll); // }, [mode]); // // Bottom sheet on zone tap: //
fetchRiskScore(zone.neighborhood_id, mode).then(setRiskScore);
```

3 — Forecast service — forecastService.ts

Used by: ForecastScreen (on mount + mode change). Makes 4 parallel calls to EC2 prediction endpoints.

src/services/forecastService.ts

```
import { apiGet } from '../utils/api'; import { USE_FAKE_DATA } from '../constants/env'; import {
fakeForecastData } from '../data/fake/fakeForecastData'; export interface HourlyPrediction { hour:
string; // e.g. '15:00' value: number; // dB, AQI, index, or count depending on category lower:
number; upper: number; health_category?: string; // AQI only } export interface ForecastData {
neighborhood_id: string; mode: 'api' | 'community'; confidence: 'low' | 'medium' | 'high';
community_report_count: number | null; // null in API mode, count in community mode generated_at:
string; noise: HourlyPrediction[]; aqi: HourlyPrediction[]; litter: HourlyPrediction[]; pollen:
HourlyPrediction[]; } // Called by ForecastScreen on mount and when mode or neighborhood changes //
Fires all 4 prediction endpoints in parallel – do not await them in sequence export async function
fetchAllForecasts( neighborhood: string, mode: 'api' | 'community' ): Promise { if (USE_FAKE_DATA)
return fakeForecastData[mode]; const params = { mode }; const [noise, aqi, litter, pollen] = await
Promise.all([ apiGet(`/predict-noise/${neighborhood}`, params),
apiGet(`/predict-aqi/${neighborhood}`, params), apiGet(`/predict-litter/${neighborhood}`, params),
apiGet(`/predict-pollen/${neighborhood}`, params), ]); // Normalize all 4 responses into one
ForecastData shape // Each endpoint returns slightly different field names – flatten here return {
neighborhood_id: neighborhood, mode, confidence: (noise as any).confidence ?? 'medium', generated_at:
(noise as any).generated_at, noise: normalizeHourly(noise, 'db'), aqi: normalizeHourly(aqi, 'aqi',
true), litter: normalizeHourly(litter, 'index'), pollen: normalizeHourly(pollen, 'count'), }; } //
Maps each endpoint's value field to a common 'value' key function normalizeHourly( raw: any, valueKey:
string, includeHealthCategory = false ): HourlyPrediction[] { return (raw.predictions ?? []).map((p:
any) => ({ hour: p.hour, value: p[valueKey], lower: p.lower, upper: p.upper, health_category:
includeHealthCategory ? p.health_category : undefined, })); } // How ForecastScreen uses this: // //
useEffect(() => { // setLoading(true); // fetchAllForecasts(selectedNeighborhood, mode) //
.then(setForecast) // .finally(() => setLoading(false)); // }, [selectedNeighborhood, mode]);
```

4 — Posts service — postsService.ts

Used by: SocialFeedScreen (feed), MapScreen (report markers), SubmitReportScreen (create), PostDetailScreen (single post), MyReportsScreen (user's own posts).

src/services/postsService.ts

```
import { apiGet, apiPost } from '../utils/api'; import { USE_FAKE_DATA } from '../constants/env';
import { fakePosts } from '../data/fake/fakePosts'; export interface ClaudeVision {
  confirmed_category: string; severity: number; description: string; } export interface Post { post_id:
  string; user_id: string; category: string; description: string; photo_url?: string; severity: number;
  lat: number; lng: number; neighborhood_id: string; agreement_count: number; petition_ready: boolean;
  petition_id?: string; claude_vision?: ClaudeVision; created_at: string; } export interface
  FeedResponse { posts: Post[]; lastKey: string | null; } // Social feed – called by SocialFeedScreen //
  aggregate=false returns individual post cards export async function fetchFeed( neighborhood: string,
  category?: string, sort: 'recent' | 'agreements' = 'recent', lastKey?: string ): Promise { if
  (USE_FAKE_DATA) return { posts: fakePosts, lastKey: null }; const params: Record = { neighborhood,
  aggregate: 'false', sort }; if (category && category !== 'all') params.category = category; if
  (lastKey) params.lastKey = lastKey; return apiGet('/posts', params); } // Report markers on the map –
  same endpoint, no aggregate export async function fetchMapReports( neighborhood: string ): Promise {
  if (USE_FAKE_DATA) return fakePosts.slice(0, 5); const res = await apiGet('/posts', { neighborhood,
  aggregate: 'false', sort: 'recent' }); return res.posts; } // Single post detail – called by
  PostDetailScreen export async function fetchPost(postId: string): Promise { if (USE_FAKE_DATA) return
  fakePosts.find(p => p.post_id === postId) ?? fakePosts[0]; return apiGet(`/posts/${postId}`); } //
  User's own posts – called by MyReportsScreen export async function fetchMyPosts(userId: string):
  Promise { if (USE_FAKE_DATA) return fakePosts.filter(p => p.user_id === userId); const res = await
  apiGet('/posts', { user_id: userId, aggregate: 'false' }); return res.posts; } // Submit a new report
  – called by SubmitReportScreen on confirm // photoUrl is the S3 URL after direct upload – pass null if
  no photo export async function createPost(data: { user_id: string; category: string; description:
  string; photo_url: string | null; severity: number; lat: number; lng: number; neighborhood_id: string;
  }): Promise { if (USE_FAKE_DATA) return { ...data, post_id: 'fake_' + Date.now(), agreement_count: 0,
  petition_ready: false, created_at: new Date().toISOString() } as Post; return apiPost('/posts', data);
  } // Agreement functions – called by ReportCard and PostDetailScreen export async function
  getAgreement( postId: string, userId: string ): Promise<{ agreement_count: number; user_has_agreed:
  boolean; petition_ready: boolean }> { if (USE_FAKE_DATA) return { agreement_count: 3, user_has_agreed:
  false, petition_ready: false }; return apiGet(`/agree/${postId}`, { userId }); } export async function
  agreePost( postId: string, userId: string ): Promise<{ agreement_count: number; user_has_agreed:
  boolean; petition_ready: boolean }> { if (USE_FAKE_DATA) return { agreement_count: 4, user_has_agreed:
  true, petition_ready: false }; return apiPost(`/agree/${postId}`, { user_id: userId }); }
```

5 — Petitions service — petitionsService.ts

Used by: PostDetailScreen (create petition), PetitionScreen (view, sign, submit).

src/services/petitionsService.ts

```
import { apiGet, apiPost } from '../utils/api'; import { USE_FAKE_DATA } from '../constants/env';
export interface Petition { petition_id: string; post_id: string; petition_text: string; official: {
  name: string; email: string; district: string }; signers: string[]; signature_count: number; status:
  'draft' | 'submitted'; created_at: string; submitted_at: string | null; } // Called by
PostDetailScreen when user taps 'Start a petition' // official is hardcoded for now – TODO: geocoding
lookup export async function createPetition(data: { post_id: string; neighborhood: string; category:
  string; official: { name: string; email: string; district: string }; }): Promise { if (USE_FAKE_DATA)
  return { petition_id: 'pet_fake', post_id: data.post_id, petition_text: 'Dear ' + data.official.name +
  ',\n\nWe the residents of ' + data.neighborhood + ' are writing to demand action on ' + data.category
  + ' issues in our neighborhood...', official: data.official, signers: [], signature_count: 0, status:
  'draft', created_at: new Date().toISOString(), submitted_at: null, }; return apiPost('/petitions',
  data); } // Called by PetitionScreen on mount export async function fetchPetition(petitionId: string):
  Promise { if (USE_FAKE_DATA) return { petition_id: petitionId, post_id: 'p_fake', petition_text: 'Dear
  Council Member...', official: { name: 'Council Member Smith', email: 'smith@council.nyc.gov',
  district: 'District 7' }, signers: [], signature_count: 0, status: 'draft', created_at: new
  Date().toISOString(), submitted_at: null }; return apiGet(`/petitions/${petitionId}`); } // Called by
PetitionScreen Sign button export async function signPetition( petitionId: string, userId: string ):
  Promise<{ signature_count: number; user_has_signed: boolean; meetup_thread_created: boolean }> { if
  (USE_FAKE_DATA) return { signature_count: 1, user_has_signed: true, meetup_thread_created: false };
  return apiPost(`/petitions/${petitionId}/sign`, { user_id: userId }); } // Called by PetitionScreen
Submit to Government button export async function submitPetition( petitionId: string, userId: string
  ): Promise<{ status: string; submitted_at: string; sent_to: string }> { if (USE_FAKE_DATA) return {
  status: 'submitted', submitted_at: new Date().toISOString(), sent_to: 'smith@council.nyc.gov' };
  return apiPost(`/petitions/${petitionId}/submit`, { user_id: userId }); }
```

6 — Users service — usersService.ts

Used by: AuthContext (on app launch), ProfileScreen (load + save), OnboardingScreen (save after signup).

src/services/usersService.ts

```
import { apiGet, apiPost, apiPut } from '../utils/api'; import { USE_FAKE_DATA } from
'../constants/env'; import * as SecureStore from 'expo-secure-store'; import { fakeUser } from
'../data/fake/fakeUser'; export interface UserProfile { user_id: string; email: string; health: {
asthma: boolean; copd: boolean; pollen_allergy: boolean; noise_sensitivity: boolean; age_group:
string; }; thresholds: { aqi: number; noise_db: number; pollen_index: number }; neighborhoods: Array<{
label: string; id: string; lat: number; lng: number }>; push_token: string | null; notification_prefs:
{ noise: boolean; air: boolean; litter: boolean; pollen: boolean; general: boolean; quiet_hours: {
start: string; end: string }; }; } // Called by AuthContext on app launch after token found in
SecureStore export async function fetchProfile(userId: string): Promise { if (USE_FAKE_DATA) return
fakeUser; return apiGet(`/users/${userId}`); } // Called by LoginScreen on signup – POST /users //
Returns user_id and JWT token – store both in SecureStore export async function createUser( email:
string, password: string ): Promise<{ user_id: string; token: string }> { const result = await
apiPost<{ user_id: string; token: string }>('/users', { email, password }); await
SecureStore.setItemAsync('jwt_token', result.token); await SecureStore.setItemAsync('user_id',
result.user_id); return result; } // Called by ProfileScreen Save button and OnboardingScreen Done
button // Only pass the fields you want to update – others stay unchanged export async function
updateProfile( userId: string, updates: Partial< > ): Promise<{ updated: boolean }> { if (USE_FAKE_DATA)
return { updated: true }; return apiPut(`/users/${userId}`, updates); } // Called on app launch after
Notifications.getExpoPushTokenAsync() export async function savePushToken( userId: string, token:
string ): Promise { if (USE_FAKE_DATA) return; await apiPost('/push-token', { user_id: userId, token
}); }
```

7 — Messages service — messagesService.ts

Used by: ThreadListScreen (list), ThreadDetailScreen (messages + send + poll).

src/services/messagesService.ts

```
import { apiGet, apiPost } from '../utils/api'; import { USE_FAKE_DATA } from '../constants/env';
import { fakeThreads, fakeMessages } from '../data/fake/fakeThreads'; export interface Thread {
  thread_id: string; type: 'dm' | 'group'; user_ids: string[]; petition_id: string | null; last_message:
  string; last_updated: string; } export interface Message { message_id: string; thread_id: string;
  sender_id: string; content: string; timestamp: string; read_by: string[]; } // Called by
ThreadListScreen on mount export async function fetchThreads(userId: string): Promise { if
(USE_FAKE_DATA) return fakeThreads; return apiGet('/threads', { userId }); } // Called by
ThreadDetailScreen on mount // Pass since to get only new messages (used by the poll) export async
function fetchMessages( threadId: string, since?: string ): Promise { if (USE_FAKE_DATA) return
fakeMessages[threadId] ?? []; const params: Record = {}; if (since) params.since = since; return
apiGet(`/messages/${threadId}`, params); } // Called by ThreadDetailScreen Send button export async
function sendMessage( threadId: string, senderId: string, content: string ): Promise { if
(USE_FAKE_DATA) return { message_id: 'msg_' + Date.now(), thread_id: threadId, sender_id: senderId,
content, timestamp: new Date().toISOString(), read_by: [senderId] }; return apiPost('/messages', {
thread_id: threadId, sender_id: senderId, content }); } // Called by PetitionScreen sign handler when
meetup_thread_created=true export async function createThread( userIds: string[], type: 'dm' |
'group', petitionId?: string ): Promise { if (USE_FAKE_DATA) return { thread_id: 'thr_fake', type,
user_ids: userIds, petition_id: petitionId ?? null, last_message: '', last_updated: new
Date().toISOString() }; return apiPost('/threads', { type, user_ids: userIds, petition_id: petitionId
?? null }); } // Polling helper – used inside ThreadDetailScreen useEffect // Returns a cleanup
function to stop polling export function startMessagePolling( threadId: string, lastTimestamp: string,
onNewMessages: (msgs: Message[]) => void, intervalMs = 10000 ): () => void { const interval =
setInterval(async () => { const newMsgs = await fetchMessages(threadId, lastTimestamp); if
(newMsgs.length > 0) onNewMessages(newMsgs); }, intervalMs); return () => clearInterval(interval); }
// How ThreadDetailScreen uses polling: // // useEffect(() => { // const lastTs =
messages[messages.length - 1]?.timestamp; // if (!lastTs) return; // return
startMessagePolling(threadId, lastTs, (newMsgs) => { // setMessages(prev => [...prev, ...newMsgs]); //
}); // }, [messages.length]);
```


8 — Photo upload — photoService.ts

Used by: SubmitReportScreen. Picks photo, uploads to S3 via presigned URL, returns the S3 URL to pass to createPost.

src/services/photoService.ts

```
import * as ImagePicker from 'expo-image-picker'; import { USE_FAKE_DATA } from '../constants/env'; //
Step 1: Pick image from camera or library export async function pickPhoto(): Promise<{ uri: string;
base64: string; } | null> { const result = await ImagePicker.launchCameraAsync({ mediaTypes:
ImagePicker.MediaTypeOptions.Images, quality: 0.7, base64: true, // needed for Claude Vision
allowsEditing: true, aspect: [4, 3], }); if (result.canceled || !result.assets[0]) return null; return
{ uri: result.assets[0].uri, base64: result.assets[0].base64 ?? '', }; } // Step 2: Get presigned S3
URL from Lambda, then upload directly to S3 // Lambda POST /posts returns a presigned_url field –
upload to that // This is called AFTER the post is created and we have the presigned URL export async
function uploadToS3( presignedUrl: string, imageUrl: string ): Promise { if (USE_FAKE_DATA) return;
const blob = await uriToBlob(imageUri); await fetch(presignedUrl, { method: 'PUT', headers: {
'Content-Type': 'image/jpeg' }, body: blob, }); } async function uriToBlob(uri: string): Promise {
const res = await fetch(uri); return res.blob(); } // Full flow used by SubmitReportScreen: // // 1.
const photo = await pickPhoto(); // 2. If litter: pass photo.base64 directly to createPost() // Lambda
calls Claude Vision internally – no separate analyze-photo endpoint // 3. const post = await
createPost({ ...data, photo_url: null }); // null until uploaded // 4. await
uploadToS3(post.presigned_url, photo.uri); // upload to S3 // 5. Lambda has already stored the final
S3 URL and Claude Vision result on the post record // 6. Navigate to ReportConfirmScreen with the
returned post – shows claude_vision fields
```

9 — AuthContext — the glue

Wraps the entire app. Loads user profile on launch. Provides user_id and profile to every screen. All services pull user_id from here — screens never read it from SecureStore directly.

src/context/AuthContext.tsx

```
import React, { createContext, useContext, useEffect, useState } from 'react'; import * as SecureStore from 'expo-secure-store'; import * as Notifications from 'expo-notifications'; import { fetchProfile, savePushToken } from '../services/usersService'; import { UserProfile } from '../services/usersService'; interface AuthContextType { userId: string | null; profile: UserProfile | null; isLoading: boolean; isAuthenticated: boolean; logout: () => Promise; refreshProfile: () => Promise; } const AuthContext = createContext({} as AuthContextType); export function AuthProvider({ children }: { children: React.ReactNode }) { const [userId, setUserId] = useState(null); const [profile, setProfile] = useState(null); const [isLoading, setLoading] = useState(true); useEffect(() => { (async () => { const storedId = await SecureStore.getItemAsync('user_id'); if (storedId) { setUserId(storedId); const p = await fetchProfile(storedId); setProfile(p); // Register push token on launch const { data: token } = await Notifications.getExpoPushTokenAsync(); await savePushToken(storedId, token); } setLoading(false); })(); }, []); async function logout() { await SecureStore.deleteItemAsync('jwt_token'); await SecureStore.deleteItemAsync('user_id'); setUserId(null); setProfile(null); } async function refreshProfile() { if (!userId) return; const p = await fetchProfile(userId); setProfile(p); } return ( { children } ); } export const useAuth = () => useContext(AuthContext); // How screens use it: // const { userId, profile } = useAuth(); // Then pass userId into service calls: // fetchFeed(profile.neighborhoods[0].id)
```

10 — Claude service — claudeService.ts

Claude calls do not go through API Gateway — they call the Anthropic API directly from the app using ANTHROPIC_API_KEY. All 4 call types from PRD section 7 are here. Screens import from this service, never call fetch() to Anthropic directly.

src/services/claudeService.ts

```
import { ANTHROPIC_API_KEY } from '../constants/env'; const CLAUDE_URL = 'https://api.anthropic.com/v1/messages'; const MODEL = 'claude-sonnet-4-20250514'; const HEADERS = { 'Content-Type': 'application/json', 'x-api-key': ANTHROPIC_API_KEY, 'anthropic-version': '2023-06-01', }; // 1 – Map zone summary (MapScreen bottom sheet) // Returns 2-3 plain English sentences about the zone's current risk export async function getZoneSummary(zone: { name: string; noise_index: number; aqi: number; health_category: string; pollen_index: number; litter_count: number; mode: string; }): Promise { const res = await fetch(CLAUDE_URL, { method: 'POST', headers: HEADERS, body: JSON.stringify({ model: MODEL, max_tokens: 256, system: 'You are an environmental health advisor. Be direct and plain. 2-3 sentences max. No jargon.', messages: [{ role: 'user', content: `Neighborhood: ${zone.name}. Noise: ${zone.noise_index}/100. AQI: ${zone.aqi} (${zone.health_category}). ` + `Pollen: ${zone.pollen_index}/100. Litter reports: ${zone.litter_count} in last 24h. ` + `Mode: ${zone.mode}. Explain the current risk to a resident in plain English.` } ] })); const data = await res.json(); return data.content[0].text; } // 2 – Forecast personal briefing (ForecastScreen Generate button) // Streaming – pass onToken callback to display text as it arrives export async function getForecastBriefing( forecast: { peak_db: number; peak_hour: string; peak_aqi: number; aqi_hour: string; pollen: number }, health: { conditions: string[]; neighborhood: string }, mode: string, onToken: (token: string) => void ): Promise { const res = await fetch(CLAUDE_URL, { method: 'POST', headers: HEADERS, body: JSON.stringify({ model: MODEL, max_tokens: 512, stream: true, system: 'You are a personal health advisor for environmental risk. Be specific, not generic. Address the user directly.', messages: [{ role: 'user', content: `Health profile: ${health.conditions.join(',`
```

```

    ')}). Neighborhood: ${health.neighborhood}. Mode: ${mode}. `+ `Forecast: noise peaks at
    ${forecast.peak_db}dB at ${forecast.peak_hour}, `+ `AQI reaches ${forecast.peak_aqi} at
    ${forecast.aqi_hour}, pollen count: ${forecast.pollen}. `+ `Give a personalized briefing for today.`
  ] ] } } ); // Stream handling – call onToken for each text chunk const reader = res.body?.getReader();
  const decoder = new TextDecoder(); while (reader) { const { done, value } = await reader.read(); if
  (done) break; const chunk = decoder.decode(value); const lines = chunk.split('\n').filter(l =>
  l.startsWith('data: ')); for (const line of lines) { try { const json = JSON.parse(line.slice(6));
  const token = json.delta?.text; if (token) onToken(token); } catch { /* skip non-JSON lines */ } } } }
  // 3 – Photo hazard analysis (SubmitReportScreen – litter category only) // Called BEFORE createPost.
  Returns structured JSON – Lambda also runs Vision server-side. // This gives the user a preview before
  they confirm submission. export async function analyzePhoto(base64Image: string): Promise<{ type:
  string; severity: number; description: string; health_impact: string; agency: string; action: string;
  }> { const res = await fetch(CLAUDE_URL, { method: 'POST', headers: HEADERS, body: JSON.stringify({
  model: MODEL, max_tokens: 256, system: 'Analyze this photo for environmental issues. Respond ONLY in
  valid JSON, no other text.', messages: [{ role: 'user', content: [ { type: 'image', source: { type:
  'base64', media_type: 'image/jpeg', data: base64Image }}, { type: 'text', text: 'Identify: type,
  severity 1-5, plain description, health_impact, agency, action.' } ] } ] }); const data = await
  res.json(); return JSON.parse(data.content[0].text); } // 4 – Petition drafting (PostDetailScreen →
  PetitionScreen) // Streaming – displays petition text token by token in editable TextInput export
  async function draftPetition( petition: { category: string; neighborhood: string; agreeing_user_count:
  number; report_texts: string[]; official_name: string; district: string; }, onToken: (token: string)
  => void ): Promise { const res = await fetch(CLAUDE_URL, { method: 'POST', headers: HEADERS, body:
  JSON.stringify({ model: MODEL, max_tokens: 1024, stream: true, system: 'You are drafting a formal
  civic petition. Tone: serious, factual, urgent but respectful. Use the reports as evidence. Address
  the official by name.', messages: [{ role: 'user', content: `Issue: ${petition.category} in
  ${petition.neighborhood}. `+ `${petition.agreeing_user_count} residents affected. `+ `Their reports:
  [${petition.report_texts.join(', ')}]. `+ `Official: ${petition.official_name}, ${petition.district}.
  Draft a petition demanding action.` } ] } ] }); // Same streaming pattern as getForecastBriefing const
  reader = res.body?.getReader(); const decoder = new TextDecoder(); while (reader) { const { done,
  value } = await reader.read(); if (done) break; const chunk = decoder.decode(value); const lines =
  chunk.split('\n').filter(l => l.startsWith('data: ')); for (const line of lines) { try { const json =
  JSON.parse(line.slice(6)); const token = json.delta?.text; if (token) onToken(token); } catch { /*
  skip */ } } } } }

```


EnviroGuard v2

Frontend API Integration

[api.ts](#) · [mapService](#) · [forecastService](#) · [postsService](#) · [petitionsService](#) · [usersService](#) · [messagesService](#) · [photoService](#) ·
[AuthContext](#)